



PROJECT DOCUMENTATION

Student Academic Resource Tracker

Setup guide, architecture, API reference and design system

Every course, every reading, every deadline.

C# / ASP.NET Core 8 Minimal API · JSON file storage · HTML / CSS / JavaScript · xUnit · Docker ·
Render

Contents

1. Overview
2. Architecture
3. Project structure
4. Prerequisites
5. Setup and run, step by step
6. Configuration reference
7. Data design
8. API reference
9. User interface and design
10. Using the application
11. Testing
12. Deployment
13. Security and design notes
14. Known limitations
15. Extending the project
16. Troubleshooting
17. Version history

1. Overview

The Student Academic Resource Tracker is a web application where a student records their courses and attaches study resources to them: lecture notes, textbooks, videos, articles, assignments, past questions and links. Each resource has a status (Not started, In progress, Completed), a priority (Low, Medium, High), an optional due date, an optional link, tags and notes.

A dashboard shows the overall completion rate, overdue and due-this-week counts, progress per course, resources by type, open items by priority, and a timeline of overdue and upcoming deadlines. All resources can be exported to a CSV file that opens cleanly in Excel.

The interface uses the University of Ghana colours (Legon navy and white) with a custom icon set, a clock-tower logo, a branded splash screen, light and dark themes, and a phone layout.

The project is a **monolith**: one ASP.NET Core application serves both the REST API and the front end, so there is a single thing to run and deploy.

Highlights

Area	What you get
Courses	Create, edit and delete courses; deleting a course removes its resources
Resources	8 types, 3 statuses, 3 priorities, due dates, links, tags and notes
Finding things	Filter by course, type, status, priority and overdue; search titles, notes and tags
Dashboard	Completion ring, status counts, deadline timeline, progress per course, library mix
Export	CSV download, safe to open in Excel
Quality	Server-side validation, crash-safe saving, 31 automated tests
Hosting	Docker image and one-click Render blueprint

2. Architecture

```

Browser (wwwroot: index.html + app.js + styles.css)
  | JSON over HTTP (/api/...)
ASP.NET Core 8 Minimal API (Program.cs)
  | DataStore.Read / DataStore.Write
JSON data file (Data/tracker.json)
  
```

Layer	Location	Role
Frontend	<code>wwwroot/</code>	Single-page UI with Overview, Resources and Courses pages, light and dark themes. Plain JavaScript calling the API with <code>fetch()</code> . Served by the same app.
Backend	<code>Program.cs</code>	Startup, REST endpoints, mapping to response objects, CSV export.

Layer	Location	Role
Persistence	<code>Services/DataStore.cs</code>	Thread-safe, all-or-nothing reads and writes of the JSON data file.
Reporting	<code>Services/AnalyticsService.cs</code>	Calculates the dashboard figures.
Validation	<code>Services/Validators.cs</code>	Checks all input and cleans up tags.
Error handling	<code>Services/BadRequestExceptionHandler.cs</code>	Turns malformed requests into readable 400 responses.
Models	<code>Models/Entities.cs</code> , <code>Models/Dtos.cs</code>	Stored entities, request bodies and response objects.
Sample data	<code>Services/SeedData.cs</code>	Three courses and eight resources created on first run.

What happens on a request

1. The browser calls an endpoint such as `POST /api/resources` with a JSON body.
2. ASP.NET Core reads the body. Malformed JSON or an unknown enum name becomes a 400 response.
3. The endpoint validates the input with `Validators` . Problems are returned as a 400 with a message per field.
4. The endpoint changes the data inside `DataStore.Write` , which holds a lock, applies the change and saves the file. If anything fails, the change is rolled back.
5. The result is mapped to a response object (for example `ResourceDto`) and returned as JSON.

Why JSON file storage

A single JSON file keeps the project easy to run anywhere: no database server, no migrations, and the data can be opened, backed up or edited by hand. All storage goes through `DataStore` , so swapping in a real database later only touches one class (see section 15).

3. Project structure

StudentResourceTracker/	
├─ Program.cs	App startup + all REST endpoints
├─ StudentResourceTracker.csproj	
├─ Models/	
│ └─ Entities.cs	Course, StudyResource, enums, DataFile
│ └─ Dtos.cs	Request and response records
├─ Services/	
│ └─ DataStore.cs	JSON persistence (locking, atomic saves, rollback)
│ └─ AnalyticsService.cs	Dashboard calculations
│ └─ Validators.cs	Input validation
│ └─ BadRequestExceptionHandler.cs	Malformed request -> 400
│ └─ SeedData.cs	First-run sample data
├─ wwwroot/	
│ └─ index.html	Layout, splash screen, dialogs and icon set
│ └─ app.js	Routing, rendering and API calls
│ └─ styles.css	Design system (UG theme, light and dark)
├─ tests/StudentResourceTracker.Tests/	xUnit unit and API tests
├─ docs/	
│ └─ DOCUMENTATION.md	This documentation
│ └─ StudentResourceTracker_Documentation.pdf	PDF version of this document
├─ Data/tracker.json	Your data (created on first run, not committed)
├─ Dockerfile	Container build used for deployment
├─ render.yaml	Render deployment blueprint
└─ README.md	Project overview and quick start

4. Prerequisites

- .NET 8 SDK
- A modern web browser (current Chrome, Edge, Firefox or Safari)
- Git, to clone the repository
- Optional: VS Code with the C# Dev Kit, Visual Studio 2022 or JetBrains Rider
- Optional: Docker, to build and run the container image locally

5. Setup and run, step by step

1. Install the .NET 8 SDK and confirm with `dotnet --version` (it should print 8.x).
2. Clone the repository: `git clone https://github.com/FelixAshong/StudentResourceTracker-c-.git StudentResourceTracker`

3. Open a terminal in the project folder (the one containing `StudentResourceTracker.csproj`): `cd StudentResourceTracker`
4. Run the app: `dotnet run` (packages are restored automatically; internet is needed the first time).
5. Open the URL printed in the terminal, for example `http://localhost:5000`.
6. Stop the app with `Ctrl + C`.

On the first run the app creates `Data/tracker.json` with sample data, so the dashboard has something to show.

From an IDE: open `StudentResourceTracker.csproj` (or the folder) in Visual Studio, VS Code or Rider and press **F5** / Run.

6. Configuration reference

The app needs no configuration to run locally. These settings change its behaviour:

Setting	How to set it	Default	Effect
Port / URL	<code>dotnet run --urls http://localhost:5080</code>	Printed at startup	Listen on a specific address
PORT	Environment variable	Not set	Listen on <code>0.0.0.0: <PORT></code> ; set automatically by Render and similar hosts
DataFile	<code>dotnet run -- DataFile=/path/to/tracker.json</code> or environment variable <code>DataFile</code>	<code>Data/tracker.json</code> in the project folder	Where the data is stored
ASPNETCORE_ENVIRONMENT	Environment variable	Production	Standard ASP.NET Core environment; <code>Development</code> shows detailed error pages

Data tasks:

Task	How
Reset to sample data	Stop the app and delete <code>Data/tracker.json</code>
Back up	Copy <code>Data/tracker.json</code>
Restore	Stop the app and copy the backup over <code>Data/tracker.json</code>

7. Data design

Data is stored as one JSON document, `Data/tracker.json`, with a list of courses and a list of resources. Identifiers are GUIDs generated by the server.

Course

Field	Type	Rules
Id	GUID	Generated by the server
Code	text	Required, max 20 characters, unique (case-insensitive), e.g. <code>CS 301</code>
Title	text	Required, max 120 characters
Lecturer	text	Optional, max 80 characters
Semester	text	Optional, max 40 characters
CreatedAt	date-time (UTC)	Set by the server

StudyResource

Field	Type	Rules
Id	GUID	Generated by the server
Courseld	GUID	Must refer to an existing course
Title	text	Required, max 200 characters
Type	enum	<code>LectureNotes</code> , <code>Textbook</code> , <code>Video</code> , <code>Article</code> , <code>Assignment</code> , <code>PastQuestion</code> , <code>Link</code> , <code>Other</code>
Url	text	Optional; must be an <code>http://</code> or <code>https://</code> address
Notes	text	Optional, max 2000 characters
Status	enum	<code>NotStarted</code> , <code>InProgress</code> , <code>Completed</code>
Priority	enum	<code>Low</code> , <code>Medium</code> , <code>High</code>
DueDate	date	Optional, <code>YYYY-MM-DD</code>
Tags	list of text	Up to 10; each trimmed, lower-cased, max 30 characters, duplicates removed
CreatedAt	date-time (UTC)	Set by the server
CompletedAt	date-time (UTC)	Set when the status becomes <code>Completed</code> , cleared if it is reopened

Relationship: one course has many resources. Deleting a course deletes its resources.

Overdue means the resource has a due date before today and its status is not `Completed`. **Due this week** means an open resource due today or in the next 6 days.

Sample data

On first run (or after deleting the data file) the app creates:

Course	Resources
CS 301 Database Systems (Dr. Mensah)	Week 3 lecture notes (completed), Assignment 2 (in progress, due in 3 days), textbook chapters 1-6 (in progress, due in 14 days), past questions 2022-2024 (due in 21 days)
CS 305 Software Engineering (Dr. Owusu)	UML tutorial videos (due in 10 days), group project proposal (2 days overdue)
STAT 201 Applied Statistics (Prof. Addo)	Hypothesis testing cheat sheet (completed), problem set 4 (due in 5 days)

Due dates are relative to the day the data is created, so the dashboard always shows a realistic mix.

How saving works

- Every read and write goes through `DataStore`, which holds a lock so concurrent requests can't interfere.
- A save writes to a temporary file and then swaps it in, so a crash mid-save can't corrupt the data file.
- Writes are all-or-nothing: if a request fails part-way, or the file can't be saved, the in-memory data is restored to the last saved version.
- Requests that change nothing (for example deleting an id that doesn't exist) don't rewrite the file.
- If the data file can't be read at startup, it is renamed to `tracker.corrupt-<timestamp>.json` and the app starts with an empty tracker, so nothing is lost.

8. API reference

All request and response bodies are JSON. Enum values are sent and returned as names (for example `"InProgress"`); numbers are rejected. Property names are case-insensitive in requests.

Method	Endpoint	Description
GET	<code>/api/courses</code>	List courses, with a resource count for each
POST	<code>/api/courses</code>	Create a course <code>{code, title, lecturer?, semester?}</code>
PUT	<code>/api/courses/{id}</code>	Update a course
DELETE	<code>/api/courses/{id}</code>	Delete a course and its resources
GET	<code>/api/resources</code>	List, filter and search resources (see below)
POST	<code>/api/resources</code>	Create <code>{courseId, title, type, status, priority, url?, notes?, dueDate?, tags?}</code>
PUT	<code>/api/resources/{id}</code>	Update a resource
PATCH	<code>/api/resources/{id}/status</code>	Change only the status <code>{status}</code>

Method	Endpoint	Description
DELETE	/api/resources/{id}	Delete a resource
GET	/api/analytics/summary	Dashboard figures
GET	/api/export/csv	Download all resources as CSV
GET	/healthz	Health check for hosting platforms, returns {"status":"ok"}

Filtering resources

All query parameters are optional and can be combined:

Parameter	Example	Matches
courseId	courseId=3bd9...	Resources in that course
type	type=Assignment	That resource type
status	status=InProgress	That status
priority	priority=High	That priority
overdue	overdue=true	Open resources with a due date before today
q	q=exam	Text in the title, notes or tags (case-insensitive)

Resources are returned open items first, then by due date (undated last), then by title. Each resource response also includes `courseCode`, `courseTitle` and `isOverdue`.

Dashboard summary

GET /api/analytics/summary returns:

Field	Meaning
totalCourses, totalResources	Counts
completed, inProgress, notStarted	Resources in each status
completionRate	Percentage of resources completed, one decimal place
overdue, dueWithin7Days	Open resources overdue, and due in the next 7 days
byCourse	For each course: code, title, total, completed and completion rate
byType	Resource count per type, largest first
openByPriority	Open resources per priority, High first
upcoming	Up to 8 open resources due from today, soonest first
overdueItems	Up to 8 overdue resources, most overdue first

CSV export

GET /api/export/csv downloads `academic-resources-YYYYMMDD.csv` with the columns `Course`, `Title`, `Type`, `Status`, `Priority`, `DueDate`, `Overdue`, `Tags`, `Url`, `Notes`, sorted by course and due

date. The file is UTF-8 with a byte-order mark so Excel shows accented characters correctly.

Responses and errors

Status	When	Body
200 / 201	Success (201 for creates, with a <code>Location</code> header)	The course or resource
204	Successful delete	Empty
400	Validation failed	<code>{"title": "One or more validation errors occurred.", "errors": {"field": ["message"]}}</code>
400	Malformed JSON or unknown enum value	<code>{"title": "The request could not be read.", "detail": "..."}"</code>
404	Unknown id	Problem details
409	Duplicate course code	<code>{"error": "A course with code 'CS 301' already exists."}</code>

Examples

```
# Create a course
curl -X POST http://localhost:5000/api/courses \
  -H "Content-Type: application/json" \
  -d '{"code":"UGRC 150","title":"Critical Thinking","lecturer":"Prof. Owusu","semester":"Semester 1"}'
```

```
# Add a resource to it
curl -X POST http://localhost:5000/api/resources \
  -H "Content-Type: application/json" \
  -d '{"courseId":"<course id>","title":"Clean Code","type":"Textbook",
    "status":"NotStarted","priority":"High","dueDate":"2026-11-15","tags":["reading"]}'
```

```
# Mark it in progress
curl -X PATCH http://localhost:5000/api/resources/<resource id>/status \
  -H "Content-Type: application/json" -d '{"status":"InProgress"}'
```

```
# High-priority overdue items
curl "http://localhost:5000/api/resources?priority=High&overdue=true"
```

9. User interface and design

Colours

The interface follows the University of Ghana colours: Legon navy with white. The navy values were chosen to match the look of the university's branding; they are not taken from an official brand guide.

Role	Light theme	Dark theme
Sidebar and splash background	Navy #002B5C fading to #001A3A	#002250 fading to #00142E
Page background	#F5F8FC	#06101F
Cards and panels	White #FFFFFF	#0B1830
Buttons, links, progress ring	#003A7A	#8DB8F2
Text	#0A1F3F	#EAF0FA

Course and resource-type colours are all shades of blue, so the page stays on brand. Red, amber and green are kept only for meaning: overdue or high priority, due soon or medium priority, and completed. Headings use the Newsreader serif for an academic feel; everything else uses Inter.

Logo and icons

- The logo is a simple clock tower inspired by the Legon campus, drawn as an SVG. It appears in the sidebar, the splash screen and the browser tab. The official University of Ghana crest is not used, because it is the university's trademark.
- All icons are a custom hand-drawn set in `wwwroot/index.html`, on a 24-pixel grid with a soft tinted fill. For example: a noticeboard for Overview, a shelf of books for Resources, a cap with tassel for Courses, and a spiral notebook, textbook with ribbon, video screen, newspaper, assignment sheet, exam paper, chain link and bookmark for the eight resource types.

Splash screen

When the app opens, a navy splash screen shows the logo, the app name, the line "Every course, every reading, every deadline." and a loading bar. It stays until the first data load finishes, shows for at least 0.9 seconds so it doesn't flash, and is forced away after 8 seconds if the server is slow (for example a sleeping free Render service).

Themes, layout and accessibility

- Dark mode:** follows the device setting on first visit; the sidebar toggle overrides it and is remembered.
- Phones:** below 860 pixels wide, the sidebar becomes a bottom navigation bar and the resource table becomes stacked cards.
- Keyboard:** `/` focuses search, `N` opens a new resource, `Esc` closes a dialog.
- Screen readers:** buttons and icons have labels, the splash screen announces "Loading your library", and notifications are announced.
- Reduced motion:** animations are switched off when the device asks for reduced motion.

10. Using the application

The app has three pages, reached from the sidebar (or the bottom bar on phones):

1. **Courses:** each course is a card showing its code, lecturer, semester and progress. Add one with *New course* (code and title are required). Hover a card to edit or delete it; deleting asks for confirmation and says how many resources will also be deleted. *View resources* opens that course's list.
2. **Resources:** add a resource with *New resource*: pick a course, enter a title and type, and optionally a status, priority, due date, link, tags (comma separated) and notes.
3. Change progress with the status pill on each row, without opening the edit form. Hover a row to edit or delete it.
4. Filter with the status tabs (which show counts), the course, type and priority menus, *Overdue only*, or the search box (titles, notes and tags). *Clear filters* resets everything.
5. *Export* (or *Export CSV* in the sidebar) downloads every resource as a spreadsheet.
6. **Overview:** a greeting with a summary sentence, a completion ring, counts for in progress, not started, due this week and overdue, a deadline timeline grouped into overdue, this week and later (click an item to edit it), progress per course, the library mix by type, and open items by priority.
7. The sidebar lists your courses; click one to jump to its resources. Toggle dark mode at the bottom of the sidebar.

A typical semester workflow: add each course at the start of the semester, add readings and assignments as they are announced, set priorities and due dates, move items to *In progress* and *Completed* as you work, and check the Overview each day for what is overdue or due this week. Before exams, filter by the `exam` tag or the *Past Question* type.

11. Testing

The `tests/StudentResourceTracker.Tests` project contains 31 xUnit tests:

- **Validators:** required fields, length limits, URL schemes, undefined enum values, tag clean-up.
- **DataStore:** seeding, persistence, rollback after a failed write, skipping unchanged saves, corrupt-file recovery.
- **AnalyticsService:** the 7-day window, overdue items not crowding out upcoming ones, completion rates.
- **API:** every endpoint, run in-process with `WebApplicationFactory` against a temporary data file.

Run them with:

```
dotnet test tests/StudentResourceTracker.Tests
```

API tests use a temporary data file, so your own `Data/tracker.json` is never touched.

12. Deployment

The app is deployed to [Render](#) as a Docker web service.

- `Dockerfile` builds the app with the .NET 8 SDK image and runs it on the smaller ASP.NET runtime image as a non-root user, storing data in `/app/Data/tracker.json`.
- `render.yaml` describes the service (free plan, health check on `/healthz`, redeploy on every push to `main`).
- Render sets the `PORT` environment variable, which the app listens on.

Deploy your own copy

1. In the Render dashboard choose **New** → **Blueprint**.
2. Connect the GitHub repository, leave the branch as `main` and the blueprint path as `render.yaml`.
3. Give the blueprint a name and click **Deploy Blueprint**.
4. Open **Resources** → **student-resource-tracker**. When the deploy shows **Live**, the public `https://...onrender.com` URL at the top of the page opens the app.

Run the container locally

```
docker build -t student-resource-tracker .
docker run -p 8080:8080 student-resource-tracker
# then open http://localhost:8080
```

To keep data between container runs, mount a folder: add `-v "$PWD/Data:/app/Data"` to `docker run`.

Free plan limitations

- The file system is not persistent, so the data resets to the sample data whenever the service restarts or is redeployed.
- The service sleeps after about 15 minutes without traffic; the first request after that takes up to a minute. The splash screen covers the wait.

For permanent data, attach a persistent disk (paid plans) and point `DataFile` at it, or replace `DataStore` with a database (see section 15).

13. Security and design notes

- All user text is HTML-escaped in the front end before it is put into the page, preventing XSS.
- Links are restricted to `http / https`, so a stored link can't run script, and open in a new tab with `rel="noopener noreferrer"`.
- CSV export prefixes cells starting with `=`, `+`, `-`, `@`, tab or carriage return with `'`, preventing spreadsheet formula injection.
- All input is validated on the server, not just in the browser.
- The container runs as a non-root user.
- There is no login. The app is intended for a single student. For multi-user use, add authentication (for example ASP.NET Core Identity) and a user id on every course and resource.

14. Known limitations

- Single user, no login.
- One JSON file holds all data, which suits hundreds or a few thousand resources rather than very large data sets.
- On the free Render plan, data resets on every restart or redeploy (see section 12).
- No file uploads; resources link to material stored elsewhere.
- No reminders or notifications for deadlines.

15. Extending the project

- **Move to a database:** replace `DataStore` with an EF Core + SQLite (or PostgreSQL) implementation. The endpoints only talk to `DataStore.Read` / `DataStore.Write`, so the change is contained.
- **Accounts:** add authentication and per-user data.
- **Uploads:** store PDFs and notes alongside each resource.
- **Reminders:** email or browser notifications for upcoming deadlines.
- **Calendar:** export deadlines as an `.ics` file for Google or Outlook Calendar.
- **Import:** load courses and resources from a CSV file.

16. Troubleshooting

Problem	Fix
<code>dotnet</code> is not recognised	Install the .NET 8 SDK and reopen the terminal.
"You must install .NET to run this application"	Install the .NET 8 SDK (not just the runtime).
Package restore fails	Check your internet connection or proxy settings.
Port already in use	Run <code>dotnet run --urls http://localhost:5050</code> .
Data looks wrong or you want to start over	Stop the app and delete <code>Data/tracker.json</code> .
A <code>tracker.corrupt-...json</code> file appeared	The data file was damaged; the original content is in that file.
Blank page	Run from the project folder so <code>wwwroot</code> is found.
Old styles after an update	Hard-refresh the browser (<code>Cmd + Shift + R</code> on Mac, <code>Ctrl + F5</code> on Windows).
Deployed site takes a long time to load	The free Render service was asleep; wait for the first request to finish.
Deployed data disappeared	Expected on the free Render plan after a restart or redeploy (see section 12).

Problem	Fix
Render shows no service under the blueprint	Open the blueprint, click Manual sync and approve creating the web service.

17. Version history

Version	Changes
Current	University of Ghana navy and white theme, custom icon set, clock-tower logo, splash screen
Previous	Redesigned front end: sidebar layout, dashboard timeline, course cards, dialogs, dark mode; Render deployment
First	Courses and resources with JSON storage, filters, dashboard, CSV export and tests

An earlier edition of this document described an Entity Framework Core and SQLite design with integer ids, statuses named To study / In progress / Done, resource types Book / Article / Video / Notes / Link / Other and a `/api/stats` endpoint. That does not match the code. The app uses JSON file storage, GUID ids, the statuses, types and priorities in section 7, and `/api/analytics/summary`. This document describes the code as it is.